FULFILLOPS

The Resilient Order-Processing Platform

Final graded DevOps capstone | Build the entire system from scratch

Audience: All students, including the two advanced students
Team size: 2-3 students (no different technical track)
Recommended effort: 40-55 total team-hours over one week, **including six supervised classroom hours** (34-49 additional team-hours). Teams must manage their workload; this is intentionally a 9.5/10 challenge.
Proposed dates: Monday 9 November to Sunday 15 November 2026, 23:59 Europe/Paris
Assessment: 100 points = 80 team points + 20 individual oral defense. The same requirements apply to every team.
Difficulty: Expert-level university capstone (target 9.5/10). No application source, Dockerfile, CI workflow, Kubernetes manifests or infrastructure code is supplied.

1. The engineering problem

FulfillOps processes customer orders for a fictional electronics fulfillment business. The public API must accept valid orders quickly, reserve stock **exactly once**, and allow a separate worker to finish orders asynchronously. Users may retry requests. Multiple clients may submit the last few items simultaneously. Infrastructure may restart during processing. A broken release must be stopped before reaching the production-like environment.

Your job is to design, implement, test, secure, containerize, deploy and operate **your own solution**. You must explain your decisions, not merely show that a program runs.

The instructor supplies a **behavioral contract** and two black-box test modes (--concurrency and --stress), not an implementation. You choose Python or JavaScript/TypeScript, your database access layer, and a durable job mechanism. PostgreSQL is the recommended shared durable store. Redis Streams is permitted but not mandatory; a transactional PostgreSQL outbox / work table is acceptable.

2. Definition of success

A satisfactory project demonstrates all of the following:

A working HTTP API that follows API_CONTRACT_EN.md exactly.

A separate worker process that completes persisted orders, with explicit retry/recovery behavior.

Atomic inventory reservation under concurrent requests, including simultaneous retries of the *same* key across API replicas; never negative stock, never double-reserve on replay.

A repeatable test suite with evidence of both failure detection and recovery.

A CI workflow: lint/static checks, tests, build, security checks, a software bill of materials (SBOM), and a gated deploy/promotion stage.

Reproducible, multistage, non-root Docker images and local Docker Compose orchestration.

Kubernetes manifests and Argo CD-based GitOps with **two API replicas and two independently operating worker instances**, persistent storage and health probes.

Evidence of observability, two distinct safe fault-injection exercises, a rolling-release verification, and an incident report.

Security discipline: secrets out of Git, least privilege, validated input, bounded request body, safe logs.

A professional README, architecture diagram, proof of contributions and individual technical defense.

3. Work packages and mandatory evidence

WP1 — Architecture and domain model

Design the system and write an architecture decision record (ADR): API, worker, durable data store and any queue. Define invariants: available >= 0; a client idempotency key maps to at most one order; the same order is never completed twice; a committed order cannot vanish if a worker restarts. Draw the data flow from HTTP request to queued order, reservation, processing, receipt and metrics.

Evidence: docs/architecture.pdf or Markdown diagram, docs/ADR-001.md, schema/migrations, threat model (at least five realistic threats).

WP2 — Contract-correct application and concurrency

Implement all mandatory routes in the contract. Validation must produce exact status codes and machine-readable error codes. Seed the three required products in a clean test database. Reserve stock **inside a database transaction** or an equivalently safe atomic mechanism. Handle multiple items, duplicate SKU lines, simultaneous requests, simultaneous requests sharing one idempotency key, and process restarts. Run the public checker with both --concurrency and --stress on a freshly seeded, isolated database. A naive in-memory counter is not acceptable.

Evidence: reproducible demo of 12+ concurrent requests for limited stock; no negative inventory; idempotent replay; no duplicated order ID; successful order completion after an API or worker restart.

WP3 — Automated quality strategy

Create unit tests, integration tests, and at least **two distinct** concurrent race tests that you own (distinct idempotency keys versus identical keys). The public checker is *not* a replacement for your own tests. Add a test that first exposes a deliberately introduced bug and then passes when the bug is repaired. Maintain test results readable in CI.

Evidence: clear local commands, test matrix by layer, screenshots/links to passing pipeline, initial failing scenario and corrected run.

WP4 — CI/CD and supply-chain integrity

GitHub Actions must run on pull requests to main: formatting/lint, unit/integration tests, black-box checker against an isolated test stack, Docker image build, dependency vulnerability scan with a documented blocking policy, **SBOM generation as a CI artifact**, and immutable artifact publication for accepted changes. Prevent a failing critical check from promoting a release. Demonstrate one **red pipeline** and the corrected **green pipeline**. Use pinned action versions or reviewed release tags, least-privilege permissions and no secrets in workflow output.

Evidence: .github/workflows/, PR link, build logs, failure recovery, and an immutable image tag (commit SHA). You do not need a paid cloud environment.

WP5 — Containers and reproducible local operations

Supply optimized multistage Dockerfiles for API and worker (or a single reusable image with distinct commands). Build as a non-root user. Use .dockerignore, minimal runtime dependencies and explicit versions. Provide compose.yaml to run the API, worker and durable store with health checks, volumes and necessary environment variables. One command should start the local stack; one command should shut it down without destroying data.

Evidence: clean rebuild, docker compose up -d --build, healthy services, container user inspection, persistence across process/container restart, and image size comparison with a naive baseline.

WP6 — Kubernetes and GitOps

Run the application in kind or another disposable local Kubernetes cluster. Provide namespace-scoped manifests (or Kustomize overlays) for at least: API Deployment (2 replicas), Worker Deployment (2 replicas, or two distinct worker pods under equivalent controller), Service, ConfigMap, Secret reference, readiness/liveness probes, CPU/memory requests and limits, and persistent database configuration. Argo CD must reconcile from a real Git repository. Demonstrate that a manual change to a Git-managed object is detected as drift and corrected; explain when automatic self-healing is safe and when human approval is required.

Evidence: infra/k8s/, Argo CD Application, screenshots/logs of Synced and Healthy, manual drift and reconciliation, rollout history, secure secret handling. Do not commit real Secret values.

WP7 — Reliability, observability and incident response

Expose liveness, readiness and Prometheus-format metrics. Track order acceptance, failures, worker retries and HTTP request volume; add meaningful latency data (histogram or equivalent) sufficient to calculate an observed p95. Propose SLO targets (example: 99.5% success over an evaluation window, p95 under 300ms in an isolated local test) and explicitly state measurement limitations. Perform **two controlled failures**: (1) kill a worker while processing a committed order, and (2) cause a failed or degraded API rolling release. Diagnose both with metrics/logs and restore service without erasing committed orders. Write a blameless postmortem.

Evidence: reproducible fault script, before/during/after health and logs, metrics output, incident timeline, root cause, mitigations, prevention action.

4. Mandatory delivery tree

This is a **target layout to create**, not a starter repository. Your structure may vary if explained in README.

```
fulfillops/
  README.md
  docs/architecture.md
  docs/ADR-001.md
  docs/threat-model.md
  docs/incident-postmortem.md
  api/              # you create all source files
  worker/           # you create all source files
  db/migrations/
  tests/
  infra/k8s/
  infra/argocd/
  .github/workflows/
  compose.yaml
  .env.example      # placeholders only, NEVER real secrets
```

A clean clone must contain enough source, dependency manifests/lockfiles, migrations, docs and scripts for a teaching assistant to reproduce the stack. Do **not** submit node_modules, virtual environments, database dumps containing personal data, credentials or private keys.

5. Advanced reliability requirements — 9.5/10 edition

These are **mandatory and graded for every team**. There is no easier track for students who did not author earlier labs. The API surface and seeded catalog in the API contract remain unchanged; the additional requirements concern operational guarantees, not new public endpoints.

A. Durable at-least-once job processing with controlled retries

Use a durable PostgreSQL work table, transactional outbox, Redis Streams with durable acknowledgement, or an equivalent architecture that can **reclaim a job after a worker dies**. Workers must atomically claim work and avoid two workers simultaneously applying the same business outcome. Implement **bounded retries** with backoff, and a persisted dead-letter / operator-review state for permanently failing jobs. An accepted order must never silently vanish. If processing exhausts retries, expose it as failed and retain the reserved stock for manual reconciliation; **do not automatically restock or fulfil twice**. Explain the exact-once business effect achieved on top of at-least-once delivery.

Required proof: start two worker instances; deliberately terminate one during processing; show the order still exists and eventually completes exactly once after recovery. Inject a deterministic poison job in the isolated lab; show its attempt count, final failed/operator-review state, and absence of an infinite retry loop. No destructive reset of a live dataset.

B. Concurrent, cross-replica idempotency

Run at least two API instances against the same persistent database. Send **eight simultaneous POST requests with the same idempotency key and same payload**; all accepted responses must return one stable order ID and must decrement stock only once. Then send competing different-key orders for the remaining inventory. Demonstrate multi-line SKU aggregation before reservation. Restart one API instance and prove that a replay is still idempotent. The supplied public checker has a --stress mode, but teams must create and explain their own race tests.

Required proof: test output, order/inventory queries, and a diagram or ADR describing the database constraint/transaction boundary. A process-local mutex does not satisfy this condition.

C. Safe progressive delivery & GitOps recovery

Use Kubernetes readiness probes and a strategy that maintains API availability during a rolling update. Demonstrate a **30-second, continuous, time-stamped health probe** while updating the API Deployment. Then inject a broken revision or readiness configuration: the rollout must be detected as unhealthy, promotion must stop, and the team must recover to a known-good Git revision using the GitOps workflow (not leave a manual cluster patch as the final state).

Required proof: deployment events, before/after Git revisions, probe log, Argo CD reconciliation evidence, no committed-order loss, and a short rationale for rollback versus fix-forward. A kind cluster is sufficient; multi-region infrastructure is not required.

D. Observable risk and supply-chain evidence

Record at least **200 local HTTP samples** for a reproducible p95 measurement and state the limitations of the local environment. CI must publish a machine-readable SBOM for the built image, run a vulnerability scan, and clearly distinguish *blocking* from *informational* findings. Document one intentional fail-to-green pipeline transition and the final immutable image digest/tag. Restrict security tests to your own lab. No paid cloud account is required.

Required proof: raw sample summary, metric query or calculation, SBOM artifact/manifest and scan logs, and the PR workflow link.

Four hard acceptance gates

A team cannot claim the corresponding advanced requirement as complete without live or reproducible evidence of: **(1)** same-key concurrency across API replicas, **(2)** worker crash recovery and bounded poison-message retries, **(3)** halted unhealthy rollout and GitOps restoration, and **(4)** a CI-produced SBOM plus a clearly enforced quality gate. These gates award points within the 80-point rubric; they are not additional hidden penalties.

6. Six supervised classroom hours and one-week execution plan

The remaining six teaching hours are **part of the project**, not additional homework. The instructor provides feedback, environment checks and assessment questions, but no completed architecture or code. Schedule the two blocks when classes meet; the dates below are indicative, not guaranteed class dates.

Block A — 3 h — Team activity: 45 min architecture + ADR; 75 min contract/API and transaction design; 60 min build/review of first race test; Instructor checkpoint: Show schema constraints, idempotency acceptance path and working baseline. Instructor challenges one edge case.

Block B — 3 h — Team activity: 60 min durable worker / retry rehearsal; 60 min CI + cluster deployment troubleshooting; 60 min fault drill, evidence review and peer challenge; Instructor checkpoint: Show one real failure and recovery, quality-gate proof and individual ownership.

After-class expectation: approximately **34–49 additional team-hours** shared among members. Work is graded on observable outcomes, not time spent. Teams should agree on roles but rotate critical ownership; each person must be able to explain the system end-to-end.

Mon 9 Nov — Checkpoint: Architecture, threat model, schema; Expected evidence: ADR, first PR and acceptance tests

Tue 10 Nov — Checkpoint: API + transactional reservation; Expected evidence: catalog, validation, safe order creation

Wed 11 Nov — Checkpoint: Durable queue, worker and race safety; Expected evidence: cross-instance idempotency and worker recovery

Thu 12 Nov — Checkpoint: Tests, CI, SBOM and scanning; Expected evidence: red-to-green PR and reproducible build

Fri 13 Nov — Checkpoint: Compose, Kubernetes and two worker pods; Expected evidence: healthy repeatable deployment

Sat 14 Nov — Checkpoint: GitOps, rollout, observability and incidents; Expected evidence: drift proof, rolling release, failure recovery

Sun 15 Nov — Checkpoint: Evidence, oral defense readiness and submission; Expected evidence: report, demo and reproducible repository

Suggested deadline: Sunday 15 November 2026 at 23:59 Europe/Paris, subject to instructor confirmation.

7. Security and acceptable-use rules

Work only on your own locally controlled lab infrastructure. Do not target real websites, production services or classmates' accounts.

No production credentials or access tokens in Git, screenshots, README, workflows or logs. Use GitHub Secrets and environment-specific configuration.

A public local metrics endpoint is acceptable *only inside the isolated lab network*; explain how it would be restricted in a production environment.

scripts/reset-test-data may operate only on a clearly labeled isolated test database. Never reset a real database.

Use your own original work. Framework boilerplate is permitted with attribution. Cite external code and any AI assistance in docs/assistance-log.md and be able to explain it.

8. Assessment rubric (100 points)

Architecture, API correctness and persistence — 12 points

Atomic concurrency, cross-replica idempotency — 16 points

Durable worker retry, dead-letter recovery and exactly-once effects — 9 points

Automated tests, red-to-green and fault injection — 10 points

CI/CD quality gates, supply-chain scan and SBOM — 9 points

Docker / Compose and runtime hardening — 6 points

Kubernetes, two-worker rollout and Argo CD GitOps — 9 points

Observability, SLO evidence and incident recovery — 5 points

Security, threat model and least privilege — 4 points

TEAM SUBTOTAL: 80 points

Individual technical defense: 20 points

TOTAL: 100 points

Each student receives the team's score **plus their own** individual defense score. The team score is not automatically reduced when one person answers poorly. The instructor may ask any student to explain a random code path, replay the public checker, describe the atomic transaction, or diagnose a broken pod.

Individual oral defense (20 points)

Architectural reasoning and tradeoffs — 5

Can explain code they contributed (live walkthrough) — 5

Diagnoses an unfamiliar failure without guessing — 5

Security, reliability, CI and GitOps reasoning — 5

9. Submission and anti-shortcut requirements

A repository URL (private repository access explicitly granted to instructor), default branch main, readable commit/PR history and contribution evidence.

A PDF report (maximum eight pages): architecture, tests, pipeline, deployment, reliability incident, limitations, credits.

A maximum 15-minute demo video or live presentation with one failing test, one successful replay, cross-replica duplicate-key and competing-key races, worker crash plus poison-message handling, a Kubernetes/GitOps rolling-recovery demo and an incident timeline.

Output of python fulfillops_public_checker.py --base-url http://localhost:8080 --concurrency --stress on a **fresh isolated test dataset**, with 15 checks passed and no skipped tests.

CI run URLs, SBOM and scan artifacts, final image digest or immutable tag; commands for installation and teardown.

Two-worker resilience and poison-job evidence, 30-second rollout CSV, and individual README note describing what you personally implemented, reviewed and would improve next.

Acceptance gates: A submission with only a design or only an API is not complete. Public checker success alone does not guarantee a high grade: CI, Kubernetes, GitOps, operations, security and individual defense are scored separately. Private tests may exercise race conditions, retry behavior and security invariants not visible in the public checker.

10. Starter materials and what you do NOT receive

The student pack contains: this brief, API_CONTRACT_EN.md, a detailed environment preparation guide, the self-contained fulfillops_public_checker.py and the local fulfillops_rollout_probe.py. No backend, worker, database schema, Compose file, Dockerfile, CI YAML, Kubernetes manifests, Argo CD configuration or instructor answers are supplied. **You build all of those.**

The public checker creates orders and consumes inventory. Use an **isolated freshly seeded test database** and reset it safely between reruns. Never point it at a production environment. Use **both** --concurrency --stress for the final validation. The checker can verify API behavior, but worker crash recovery, GitOps and CI must also be demonstrated separately.

11. Official references

Docker documentation: https://docs.docker.com/X

GitHub Actions: https://docs.github.com/en/actionsX

Kubernetes documentation: https://kubernetes.io/docs/home/X

kind quick start: https://kind.sigs.k8s.io/docs/user/quick-start/X

Argo CD getting started: https://argo-cd.readthedocs.io/en/stable/getting_started/X

Prometheus metric exposition: https://prometheus.io/docs/instrumenting/exposition_formats/X

All implementation decisions remain the team's responsibility. The instructor will assess the results against the contract, invariants and evidence—not against a particular framework.